

Nomes: Lucas Vargas e José Gabriel
Disciplina: Matemática Computacional / Noite

Objetivo do Trabalho

Compreender, pesquisar e aplicar conceitos fundamentais sobre **árvores em computação**, relacionando teoria e prática.

Parte 1 — Conceitos Teóricos

Explique com suas palavras:

1. Definição de Árvore

- O que é uma árvore em computação?
Uma árvore em computação é uma estrutura de dados hierárquica composta por nós conectados entre si. Ela possui um nó principal chamado raiz, do qual se originam outros nós chamados filhos. Cada nó filho também pode possuir seus próprios filhos, formando diferentes níveis hierárquicos dentro da estrutura. Os nós que não possuem filhos são chamados de folhas.
Uma característica importante das árvores é que elas não permitem relações cíclicas, ou seja, um nó descendente não pode apontar de volta para um nó ancestral, evitando ciclos como “um neto apontar para o avô”.

Esse tipo de estrutura é amplamente utilizado para representar informações hierárquicas, como sistemas de arquivos e pastas, organogramas, árvores genealógicas e estruturas de bancos de dados.
 - Diferença entre árvore e grafo
O grafo pode existir relação cíclica e a árvore não.
-

2. Componentes de uma Árvore

Descreva os conceitos:

- Nó - Componente/Objeto da árvore.
 - Raiz - Nó original da árvore
 - Folha - Nós na última camada da árvore
 - Aresta - Ligamento entre dois nós.
 - Subárvore - Um nó, que não seja o nó raiz, e seus descendentes que participam de uma outra árvore.
 - Altura - É o número máximo de níveis (ou camadas) que uma árvore possui. Também pode ser definida como a maior distância entre a raiz e uma folha.
 - Profundidade - É a quantidade de arestas existentes entre um determinado nó e a raiz da árvore. Quanto mais distante da raiz, maior será sua profundidade, o mais distante possível é igual a altura da árvore.
-

3. Tipos de Árvores

Explique os seguintes tipos:

- Árvore binária → Estrutura de dados hierárquica e não linear, composta por nós, onde cada nó pode ter, no máximo, dois nós filhos, chamados de filho da esquerda e filho da direita.

- Árvore binária de busca (BST) → Estrutura de dados hierárquica baseada em nós, onde cada nó contém uma chave e no máximo dois filhos (esquerdo e direito).
 - Árvore balanceada (AVL ou Red-Black) → Estrutura de dados de árvore que mantém automaticamente sua altura pequena, limitando a diferença de profundidade entre subárvores à esquerda e à direita, geralmente em no máximo 1.
 - Heap (mínimo ou máximo) → Estrutura de dados baseada em árvore, que organiza elementos com base em prioridade, permitindo acesso rápido ao valor máximo ou mínimo. É considerada uma árvore binária quase completa, onde cada nó pai é maior (Max-Heap) ou menor (Min-Heap) que seus filhos, facilitando a implementação de filas de prioridade.
-

Parte 2 — Operações em Árvores

Explique e exemplifique:

1. Inserção

Em uma BST:

- valores menores ficam à esquerda;
- valores maiores ficam à direita.

Inserindo os valores: 50, 30, 70, 20 e 40:

```

      50
     /  \
    30   70
   /  \
  20  40

```

- 30 foi inserido à esquerda de 50.
- 70 foi inserido à direita de 50.
- 20 ficou à esquerda de 30.
- 40 ficou à direita de 30.

2. Busca

A busca consiste em localizar um valor dentro da árvore.

Exemplo

Buscar o valor 40:

1. Começa na raiz (50).
2. 40 é menor que 50 → vai para a esquerda.
3. Chega em 30.
4. 40 é maior que 30 → vai para a direita.
5. Encontra 40.

50 → 30 → 40

A busca em árvores binárias de busca é eficiente porque elimina metade das possibilidades a cada comparação.

3. Remoção (conceito geral)

A remoção consiste em excluir um nó da árvore sem quebrar sua estrutura.

Existem 3 casos principais:

a) Nó folha

O nó não possui filhos.

Exemplo: remover 20.

Antes:

30
/
20

Depois:

30

b) Nó com um filho

O filho ocupa o lugar do nó removido.

Antes:

30
\
40

Depois:

40

c) Nó com dois filhos

Substitui-se o nó por:

- o maior valor da subárvore esquerda;
ou
- o menor valor da subárvore direita.

Exemplo:

```
50
 / \
30  70
```

Ao remover 50, ele pode ser substituído por 70 ou por outro valor adequado da árvore.

4. Percursos (Traversals)

- Pré-ordem
Primeiro visita a raiz, depois a esquerda e por último a direita.
50 → 30 → 20 → 40 → 70
- Em ordem
Visita primeiro a esquerda, depois a raiz e por último a direita.
20 → 30 → 40 → 50 → 70
- Pós-ordem
Visita esquerda, direita e por último a raiz.
20 → 40 → 30 → 70 → 50

Parte 3 — Aplicação Prática

Simulação Manual

Construa manualmente uma árvore inserindo os valores:

```
50, 30, 70, 20, 40, 60, 80
```

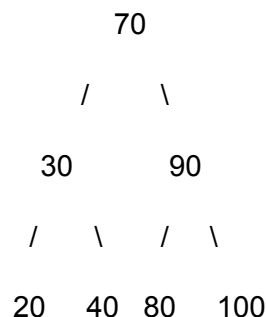
Depois:

- Desenhe a árvore
- Mostre os seguintes percursos:
 - Em ordem
 - Pré-ordem
 - Pós-ordem

Opção B: 1. Construção Passo a Passo

1. **70**: Raiz.
2. **30**: Menor que 70, vai para a **esquerda**.
3. **90**: Maior que 70, vai para a **direita**.
4. **20**: Menor que 70 é menor que 30, vai para a **esquerda do 30**.
5. **40**: Menor que 70 é maior que 30, vai para a **direita do 30**.
6. **80**: Maior que 70 é menor que 90, vai para a **esquerda do 90**.
7. **100**: Maior que 70 é maior que 90, vai para a **direita do 90**.

2. Desenho da Árvore



3. Percursos

Os percursos são formas diferentes de visitar todos os nós da árvore:

- **Em-ordem (In-order)**: Visita a subárvore esquerda, a raiz e a subárvore direita. Em uma BST, o resultado é sempre os valores em ordem crescente.
 - **Resultado**: 20, 30, 40, 70, 80, 90, 100
- **Pré-ordem (Pre-order)**: Visita a raiz primeiro, depois a subárvore esquerda e a subárvore direita.
 - **Resultado**: 70, 30, 20, 40, 90, 80, 100
- **Pós-ordem (Post-order)**: Visita a subárvore esquerda, a direita e por último a raiz.
 - **Resultado**: 20, 40, 30, 80, 100, 90, 70

Parte 4 — Análise Computacional

Responda:

1. Qual a complexidade da busca em:
 - Árvore binária comum - no pior caso, a busca pode precisar percorrer nó por nó, especialmente se a árvore estiver desbalanceada (parecendo uma lista).
Complexidade: $O(n)$
 - Árvore balanceada - A cada passo, metade das possibilidades é descartada, pois a árvore mantém sua estrutura equilibrada.
Complexidade: $O(\log n)$
2. O que acontece quando uma árvore fica desbalanceada?

Quando uma árvore fica desbalanceada, alguns lados passam a ter muito mais nós do que outros. Isso faz com que a árvore perca eficiência, podendo se comportar como uma lista encadeada.

Consequências

1. aumento da altura da árvore;
 2. buscas mais lentas;
 3. inserções e remoções menos eficientes;
 4. complexidade pode passar de $O(\log n)$ para $O(n)$
-
3. Por que árvores são importantes em computação?

As árvores são importantes em computação porque permitem organizar dados de forma hierárquica e eficiente, facilitando operações como busca, inserção e remoção.

Parte 5 — Conexão com Indução

Explique:

- Como a indução matemática pode ser usada para provar propriedades de árvores

Exemplo:

Uma árvore com n nós possui $n - 1$ arestas

A indução matemática é uma ferramenta natural para provar propriedades de árvores porque árvores possuem uma **estrutura recursiva**: toda árvore pode ser construída a partir de árvores menores (subárvores) ou removendo-se folhas/nós de uma árvore maior. Isso permite dois estilos comuns de indução:

Indução sobre o número de nós (n): demonstra-se a propriedade para a árvore mínima (geralmente $n = 1$, um único nó) e supõe-se válida para toda árvore com k nós, mostrando então que continua válida para árvores com $k + 1$ nós.

Indução estrutural: prova-se a propriedade para a árvore base (folha) e, assumindo que vale para as subárvores de uma raiz, mostra-se que vale para a árvore inteira.

Em ambos os casos, a chave é que remover uma folha (ou separar uma subárvore) produz uma árvore menor, sobre a qual a hipótese de indução pode ser aplicada.

Parte 6 — Aplicações Reais

Cite e explique pelo menos 3 aplicações de árvores:

Exemplos:

- Sistemas de arquivos
- Banco de dados (indexação)
- Árvores de decisão
- Compiladores
- Inteligência artificial

Sistemas de arquivos:

Em sistemas de arquivos, as **árvores** são a estrutura fundamental para organizar dados, permitindo que o sistema operacional encontre, leia e grave arquivos de forma eficiente. O conceito se manifesta em dois níveis principais: a organização lógica (como você vê as pastas) e a organização física (como os dados são indexados no disco).

Aqui está uma visão detalhada de como essa estrutura opera:

1. Organização Lógica: A Árvore de Diretórios

Para o usuário, o sistema de arquivos é uma **Árvore N-ária**. Ela começa em um **nó raiz** (o `/` no Linux ou `C:\` no Windows) e se ramifica em diretórios e subdiretórios.

- **Nós Internos:** São as pastas (diretórios), que contêm referências para outros nós.
- **Folhas:** São os arquivos reais, onde o caminho termina.
- **Caminhos (Paths):** Representam a travessia da árvore da raiz até a folha (ex: `/home/usuario/documento.txt`).

2. Organização Estrutural: B-Trees e B+ Trees

Internamente, a maioria dos sistemas de arquivos modernos (como **NTFS**, **EXT4**, **XFS** e **APFS**) não utiliza árvores binárias simples, mas sim variações chamadas **B-Trees** ou **B+ Trees**.

Essas estruturas são ideais para discos porque:

- **Baixa Altura:** Elas mantêm a árvore "achatada", o que significa que o sistema precisa realizar pouquíssimas leituras de disco para encontrar um arquivo, mesmo em volumes com milhões de itens.
- **Ordenação:** Os nomes de arquivos ou identificadores (Inodes) são mantidos ordenados, permitindo buscas binárias rápidas $O(\log n)$.
- **Balanceamento:** A árvore se autoajusta conforme arquivos são criados ou deletados, garantindo que o tempo de acesso seja constante.

3. O Papel dos Inodes e Extents

Em sistemas como o **EXT4**, a "árvore" também gerencia onde os blocos de dados físicos estão no HD ou SSD:

- **Árvore de Extents:** Em vez de listar cada bloco individual de um arquivo grande, o sistema usa uma árvore para mapear "faixas" de blocos contínuos. Se um arquivo ocupa do bloco 100 ao 500, a árvore armazena apenas essa faixa, economizando espaço de metadados.
- **Resolução de Conflitos:** Árvores permitem que o sistema gerencie permissões, datas de modificação e links físicos de forma hierárquica e rápida.

Por que usar Árvores?

Sem essa estrutura, para encontrar um arquivo em um disco de 1TB, o computador teria que fazer uma busca linear (olhar arquivo por arquivo), o que levaria segundos ou minutos. Com a estrutura de **B+ Tree**, o sistema encontra qualquer arquivo em uma fração de milissegundo, geralmente em 3 ou 4 "saltos" de leitura.

Inteligência artificial:

Na Inteligência Artificial, árvores são utilizadas para representar possibilidades, decisões e caminhos que um algoritmo pode seguir para resolver um problema.

Exemplos de uso

- **Árvores de decisão:** utilizadas em machine learning para classificação e tomada de decisões. Cada nó representa uma pergunta ou condição, e cada ramo representa uma possível resposta.

Exemplo:

Chove?

/ \

Sim Não

/ \

Levar guarda- Sair sem

chuva guarda-chuva

- Jogos e busca de estratégias: algoritmos de IA utilizam árvores para analisar possíveis jogadas em jogos como xadrez e dama. Cada nível da árvore representa uma nova possibilidade de movimento.
- Sistemas especialistas: árvores ajudam a organizar regras e decisões para auxiliar diagnósticos e recomendações.

Banco de dados(indexação)

Em bancos de dados, árvores são utilizadas para organizar e localizar informações de forma rápida e eficiente. Isso é feito por meio de estruturas chamadas índices, que funcionam de maneira semelhante ao índice de um livro.

Como funciona

Os dados são organizados em árvores balanceadas, como árvores B e B+, permitindo que o sistema encontre registros sem precisar percorrer toda a tabela.

Exemplo

Imagine uma tabela com milhares de usuários cadastrados. Ao procurar um usuário pelo CPF, o banco utiliza a árvore de indexação para reduzir rapidamente o número de registros analisados.

Em vez de verificar um por um:

001

002

003

004

...

o banco divide a busca em partes menores até encontrar o valor desejado.